

[ASRJC/2025/Q1]

1 Name your Jupyter Notebook as:

TASK1_<your name>_<centre number>_<index number>.ipynb

Token-based short run encoding is a compression technique that replaces consecutive repeated characters with a token followed by the character and the count of repetitions.

Unlike run-length encoding, token-based short run encoding only applies compression when there are 3 or more consecutive identical characters, leaving shorter sequences unchanged for better efficiency.

For example, the string

aaaaaaaaaaaaabbcccd

would be encoded as

%a14bb%c3d

where "%" is the token, followed by the character and its count.

The task is to write the encoding and decoding logic for this encoding method.

For each of the sub-tasks, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program code
```

Output:

Task 1.1

Write a function `encode_str(text, token)` to convert a long string into its shortened version using token-based short run encoding with a specified token character and return the encoded string.

If `text` contains the token character or digit characters, the function should instead return a meaningful error message.

Test your function with suitable test cases. [5]

Task 1.2

Write a function `decode_str(encoded, token)` to convert an encoded string back to its original form and return this decoded string. You may assume that the encoded string is valid.

Test your function with the valid output strings you obtained in Task 1.1 to verify the original strings are recovered. [5]

Task 1.3

Write program code to:

- read the file `encoding.txt`, which contains encoded strings
- decode each line and output the resulting ASCII art in the notebook, given that the token is "@".

This ASCII art should resemble something familiar. [3]

Save your Jupyter Notebook for Task 1.